

Smurfs: Multi-Agent System using Context-Efficient DFSDT for Tool Planning

Junzhi Chen*, Juhao Liang*, Benyou Wang[✉]

The Chinese University of Hong Kong, Shenzhen

Shenzhen Research Institute of Big Data

wangbenyou@cuhk.edu.cn

Abstract

Teaching large language models (LLMs) to use tools for solving complex problems can grant them human-like reasoning abilities. ReAct and its variants are popular frameworks for tool use in both single-agent and multi-agent systems. To address issues like error propagation and limited exploration in ReAct, the Deep First Search Decision Tree (DFSDT) was proposed, but it faces challenges such as rollback instability, redundant context, and premature termination in single-agent settings. We introduce "Smurfs," a novel multi-agent system (MAS) that enhances DFSDT with a modular, context-efficient, and training-free design. Smurfs surpasses baseline methods in both the open-ended StableToolBench and the closed-ended HotpotQA tasks, reducing token usage by 60.9% compared to DFSDT and enabling Mistral-7b to perform on par with GPT-4-DFSDT. Extensive ablation studies confirm the effectiveness of Smurfs' core components, offering valuable insights for the construction and interpretation of MAS, and paving the way for future exploration. We release the code at <https://github.com/FreedomIntelligence/Smurfs>.

1 Introduction

The ability to manipulate tools for complex tasks has long been considered a distinctive characteristic of humans (Oakley and Museum, 1972; Ambrose, 2001). Can we extend this ability to today's large language models (LLMs), enabling them to utilize multiple tools to perform complex tasks beyond their inherent capabilities? If LLMs can use external tools to access knowledge or execute tasks beyond their fixed language modeling capabilities, we can shift the focus of LLM training towards enhancing their reasoning and tool-use skills. This

shift would allow tools to supplement what LLMs should know or execute, thereby improving the parameter efficiency of the LLMs.

	Pass Rate ↑ (%)	Win Rate ↑ (%)	# of Tokens per request ↓	# of Tokens per query ↓
ReAct	44.4 $\pm$ 1.1	<i>base</i>	1,424	6,479
DFSDT	55.4 $\pm$ 2.0	60.4	1,743	20,714
Smurfs (ours)	57.4 $\pm$ 1.1	62.4	459	8,096

Table 1: Comparison of token cost and performance between tool planning methods over StableToolBench. Existing methods, *ReAct* and *DFSDT*, have limitations due to high token costs or poor performance. The results are averaged over the subtasks within StableToolBench.

In previous multi-agent systems for tool planning, methods like Chain-of-Thought (CoT) (Wei et al., 2023), ReAct (Yao et al., 2022), and the more advanced DFSDT (Qin et al., 2024) have been proposed to enhance LLMs' ability to handle complex multi-step tasks. However, these approaches face notable limitations. ReAct has trouble in handling *error propagation* and *limited exploration*. DFSDT incorporates a rollback mechanism and depth-first search approach to address limitations of ReAct, but it suffers from *instability* when the base model struggles with long-context reasoning. It also introduces inefficiencies due to *redundant context* handling and risks *premature termination* when solving multi-step problems. These challenges highlight the need for further innovation to better manage context and reasoning complexity in multi-tool planning systems.

In this paper, we introduce '*Smurfs*', an innovative multi-agent system (MAS) framework inspired by the collaborative and versatile nature of its namesake cartoon characters. The proposed framework leverages enhanced DFSDT to perform complex tool planning tasks without the need for additional training. The effectiveness of Smurfs is demonstrated through both open-ended and closed-ended tool planning benchmark experiments (Guo et al., 2024; Yang et al., 2018), where it consis-

*These authors contributed equally to this work.

✉Corresponding author.

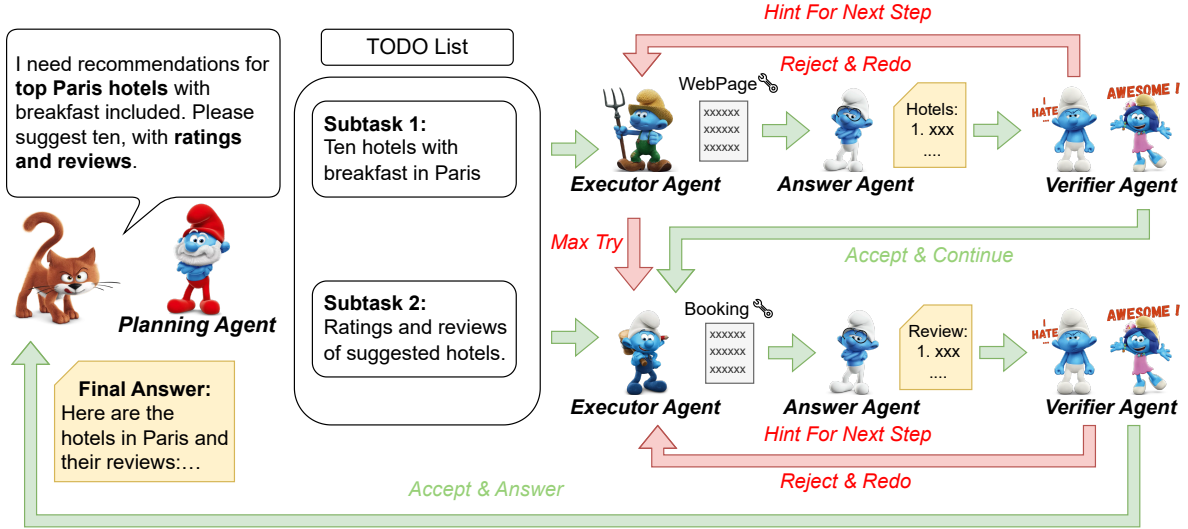


Figure 1: Demonstration of the whole process of the Smurfs framework.

tently outperforms baseline methods. An ablation study, followed by a case study, further investigates the reasons behind this effectiveness. These results establish a new state-of-the-art in the field and provide concrete evidence of the advantages of a multi-agent approach in enhancing LLM capabilities.

The contributions of this paper can be summarized as follows:

1. We introduce a highly modular, context-efficient, and training-free MAS framework that utilizes an enhanced DFSDT to improve the tool planning capabilities of LLMs. Experiments demonstrate the effectiveness of this approach, which also proves to be more cost-efficient compared to existing methods.
2. Through ablation studies, we uncover the underlying factors contributing to the effectiveness of the MAS framework, offering valuable insights for future research.

2 Related Work

To augment LLMs to do multi-tool planning for solving complex problems, previous work has seen numerous attempts. Chain-of-Thought (Wei et al., 2023) is the first to propose the method of thought and answer chain reasoning. ReAct (Yao et al., 2022) further introduce the thought-action-observation format for tool chain reasoning, leading to the development of various multi-tool planning methods (Chen et al., 2023a; Xu et al., 2023; Shinn et al., 2023). The latest work,

Deep First Search Decision Tree (DFSDT) (Qin et al., 2024), is proposed to address the inherent limitations of CoT and ReAct: **(1) Error Propagation:** Error occurs at early stage of planning will result in wrong answer in the end, but it can only be identified until reaching the end of the planning chain **(2) Limited Exploration:** Single solution chain can't explore the planning space completely.

DFSDT is powerful in addressing multi-tool planning problems. Its core concept involves employing a depth-first search (DFS) approach for multi-tool planning and backtracks whenever an LLM think the solving process has entered a wrong state (for more details, see Appendix B). When a tool fails or is deemed inadequate for solving the current problem, DFSDT backtracks to the previous solution state and attempts to resolve the issue using another solution plan. However, three limitations are identified with the mechanism of DFSDT:

Limit I: Instability of the Rollback Mechanism

The rollback mechanism in DFSDT is determined by the model. The number of steps to roll back and the selection of new tools after rollback are guided using prompt containing the errors encountered in the previous failed trajectory. When the model is sufficiently robust, this rollback mechanism serves as a highly flexible and efficient planning strategy. However, when the model's capability is insufficient, it will fail to execute the correct rollback mechanism, i.e. retry the same error tools or roll

back too far.

Limit II: Redundant Context In the process of planning with DFSDT, each tool plan is generated using the entire conversation history (including all the thoughts, actions, action inputs and tool responses) as context. However, in reality, each step of tool planning only requires a very small portion of the relevant history for effective planning.

The context redundancy not only increases computational overhead but also reduces the accuracy of model inference due to the inclusion of irrelevant historical data. As highlighted by (Liu et al., 2024), redundant context become particularly noticeable in tasks requiring assimilation and processing of large inputs, like verbose tool documents and API responses. The situation worsens when LLMs are supplemented with external information, such as document retrieval or online searching (Petroni et al., 2020; Ram et al., 2023; Mallen et al., 2022). Although numerous language models capable of handling larger contexts are emerging (Dai et al., 2019; Dao et al., 2022), they often face significant performance degradation when the important information is located at some positions (Liu et al., 2024; Shi et al., 2023), which is known as the ‘*lost-in-the-middle*’ problem.

Limit III: Premature Termination The termination mechanism set by DFSDT involves adding a termination tool to the model’s selectable toolkit. When the model selects this termination tool, DFSDT stops and provides an answer. However, in practical applications, this mechanism often prematurely terminates when dealing with complex problems requiring multi-step reasoning. We hypothesize that this issue arises due to the redundant interference of other tool information and history information, which disrupts the model’s ability to judge whether the original problem should be terminated. Instead, the model focuses on whether the current sub-problem requires termination, leading the mechanism to terminate after resolving the sub-problem.

In conclusion, DFSDT relies highly on the base model’s reasoning ability, especially long context reasoning ability to make roll back decision, termination decision and tool choice decision at the same time, which is a very difficult task even with the most powerful LLM like GPT4 (Yuan et al., 2024).

Multi-Agent System To address the limitations inherent in DFSDT and to further enhance LLM’s multi-tool planning capabilities, MAS has emerged as a natural solution. Inspired by human social division of labor and cooperation, MAS aim to enable AI agents to accomplish more complex tasks through the division of labor and collaboration. By decomposing the task of DFSDT to multiple agents and giving them only the information they need, we can enable LLMs to use DFSDT more effectively and more efficiently.

Method	Multi-Agent	Training	Generality	Reflection	Planning
ReACT (Yao et al., 2022)	✗	✗	✓	✗	Iterative
Reflexion (Shinn et al., 2023)	✗	✗	✓	✓	Iterative
Chameleon (Lu et al., 2023)	✗	✗	✓	✗	Global
HuggingGPT (Shen et al., 2023)	✗	✗	✓	✗	Global
BOLAA (Liu et al., 2023)	✓	✗	✓	✗	Iterative
AgentVerse (Chen et al., 2023b)	✗	✗	✓	✓	Iterative
FIREACT (Chen et al., 2023a)	✗	✓	✗	✓	Iterative
DFSDT (Qin et al., 2024)	✗	✗	✗	✗	Iterative
RESTGPT (Song et al., 2023)	✓	✗	✗	✗	Iterative
Lumos (Yin et al., 2024)	✓	✓	✗	✗	Iterative or Global
AutoAct (Qiao et al., 2024)	✓	✓	✗	✓	Iterative
Smurfs (Ours)	✓	✗	✓	✓	Iterative and Global

Table 2: Comparison of tool use systems.

Previous work on multi-agent system mainly focus on coding and society simulation area (Hong et al., 2023; Qian et al., 2024; Park et al., 2023; Li et al., 2023; Wu et al., 2023). For tool-use scenario, most multi-agent systems still use the ReAct style reasoning (Qiao et al., 2024; Chen et al., 2023b; Yuan et al., 2024; Liu et al., 2023; Song et al., 2023; Yin et al., 2024; Xu et al., 2023), only using multi-agent discussion and revision to increase reasoning quality, which still inherits the limitation of ReAct. Therefore, this paper aims to construct a novel MAS framework to address the aforementioned limitations. Table 2 shows the difference between different tool-use systems. More detailed discussion can be seen in Appendix A.

Token Compression Token compression refers to compressing tokens fed into LLMs while preserving inference performance, thus reducing computational overhead and mitigating the constraints imposed by long context limits. Previous works (Mu et al., 2024; Fu et al., 2024) have explored token compression techniques, with a focus on compressing token embeddings. In contrast, Smurfs achieves context compression by filtering the input context for each tool planning process. Table 6 shows the detailed difference between different compression techniques.

In the future, these compression methods can be used together in multi-tool planning scenarios to achieve more efficient token compression. Each agent’s system prompt could be compressed using gist tokens (Mu et al., 2024), tool descriptions

could be managed with CAMPHOR’s compression approach (Fu et al., 2024), and the context input by each agent could be compressed using Smurfs.

3 Smurfs: MAS with Context Efficient DFSDT

The Smurfs, the beloved cartoon characters, symbolize unity and resourcefulness, and are good at using tools to overcome any challenge they encounter.

3.1 Framework Overview

Figure 1 illustrates the entire workflow for the Smurfs framework. Initially, the **Planning Agent** identifies the user’s complex query and breaks it down into manageable sub-tasks. **Executor Agents** are then tasked with collecting task specific information, utilizing access to external tools. **Answer Agent** compiles the findings into a cohesive response, which is subsequently verified by the **Verifier Agent** to ensure accuracy and relevance.

By dividing tasks among different agents, each agent can focus on a specific part of the task, accessing only the necessary history as context during task execution, which effectively addresses the issue of **redundant context**. The redesign of the rollback mechanism to incorporate memory and tool list rollback mechanisms addresses the **instability of the rollback mechanism**. Drawing on the concept of least-to-most prompting (Zhou et al., 2023), the original problem is first decomposed into sub-problems for macro-level planning. Subsequently, Smurfs is used to solve each sub-problem at the micro-level, with macro-level planning guiding the micro-level planning, thereby resolving the issue of **premature termination**.

In the rest of this section, the mechanism of the system and the functions of each agent will be detailed. More details of memory system can be seen at Appendix C.

3.2 Agent Components

In this section, we introduce the two core components of the Smurfs system:

Tools The tool documents about the tools that Smurfs can utilize in the completion of a complex task are denoted as $D = \{n_i, d_i, p_i\}_{i=1}^{|d|}$, where n represents the tool name, d represents tool usage description, p represents parameter description and $|d|$ represents the amount of the available tools. The

available tool list is denoted as $\tau = \{n_i, d_i\}_{i=1}^{|\tau|}$. τ_t denotes the tool list Smurfs can utilize at time t .

Memory The memory of the agent system at time t is the history of the task-solving process before t , denoted as $M = (m_1, m_2, \dots, m_{t-1})$ and $m_i = (\gamma_i, a_i)$, where m_i represents memory element at time i and γ_i, a_i represents thought and answer generated by the system at time i . There are two types of memory in Smurfs: local memory and global memory. the local memory is used to record the ongoing solution trajectory and to generate the next action in the current trajectory. The global memory, meanwhile, records all trajectories and is used to generate the sub-problem’s answer by combining all trajectory records when the maximum number of retries is exceeded. This local-global combined memory system ensures that the planning of the current solution trajectory is not influenced by the context of erroneous trajectories. It also generates an answer that combines all trajectories when the verifier agent cannot determine task completion within the maximum number of planned steps. This memory system ensures context efficiency during the task-solving process.

3.3 Macro Planning

Planning Agent The primary responsibility of the Planning Agent is doing macro-level planning through task decomposition to prevent **premature termination**. The inference process of the Planning Agent is:

$$Plan P : (p1, p2, \dots) = PA(q) \quad (1)$$

Where p_i represents sub-problem of the original query q , PA represents the Planning Agent. After the task decomposition, the agent system will use Executor Agent, Answer Agent and Verifier Agent to solve each sub-problem using DFSDT collaboratively in a sequential order. To utilize the answer of the previous sub-problem when solving subsequent sub-problem, the strategy known as least-to-most prompting (Zhou et al., 2023) is used.

3.4 Subtask Solving Process

After introducing the function of plan agent, this section outlines how the agents collaborate to solve sub-tasks, as shown in Figure 2.

Stable Rollback To address the **instability of the rollback mechanism** in DFSDT, we propose a rollback mechanism based on rules. Whenever

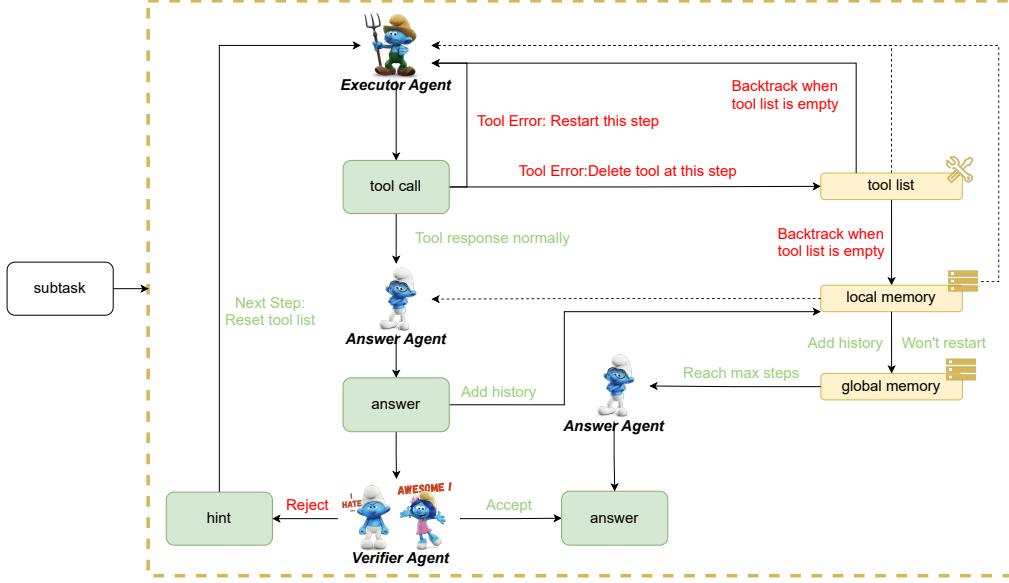


Figure 2: Details of the subtask-solving process of the Smurfs framework. The dotted line represents that the agent can see the memory and the full line stands for operation.

an error occurs while using a tool $\tau_{t,i}$ at time t , the tool list at t τ_t will pop $\tau_{t,i}$ out and reperform tool selection and tool planning (ensuring that the faulty tool is not selected again). If, at time t , the tool list becomes empty, it signifies that after the system choosing tool $\tau_{t-1,j}$ at time $t-1$, no subsequent trajectory can solve the problem. In this case, the agent system will roll back to time $t-1$, meaning that the local memory M will pop out the memory element m_{t-1} at time $t-1$, and the tool list at time $t-1$ τ_{t-1} will pop out tool $\tau_{t-1,j}$. The agent system will then set the time $t=t-1$ and continue planning. This rule-based rollback mechanism, compared to the original model-based rollback mechanism of DFSDT, is less flexible and might reduce rollback efficiency. However, it is more stable, ensuring the correctness of deep first search and enabling models with weaker capabilities to utilize DFSDT on tool planning.

Executor Agent The Executor Agent is responsible for choosing and executing the tools to solve the sub-tasks. At each time t , the agent can invoke one tool to tackle the given task:

$$\gamma = EA.gen_thought(p, M, \tau, h) \quad (2)$$

$$\alpha = EA.choose_tool(p, \gamma, \tau) \quad (3)$$

$$\beta = EA.gen_arguments(p, M, D[\alpha]) \quad (4)$$

$$r = EA.call_tool(\alpha, \beta) \quad (5)$$

Where p is the sub-problem from Planning Agent, h is the hint from the Verifier Agent, τ is the tool

list, M is local memory, $D[\alpha]$ means the tool document of tool α . The agent, using the ReACT format (Yao et al., 2022) to choose the tool and arguments, then execute the tool. Noticed that each inference process only uses the relevant part from the local memory and tool list to reduce the context redundancy. More detailed information of the Executor Agent can be found in Figure 5.

Answer Agent To mitigate the performance degradation caused by lengthy contexts, we introduce the Answer Agent role, designed to extract crucial content for each step and sub-problem:

$$Answer : a = AA(q, r, M) \quad (6)$$

Where q is sub-problem from the Planning Agent, r is response from the Executor Agent, M is the local memory (or global memory if max retry reaches). As the ‘lost-in-the-middle’ theory described in section 1, retaining all information may not always be beneficial, particularly in cases where the solution path is challenging to discern. Therefore, the primary role of the Answer Agent is to succinctly summarize the generated answers and tool responses to maintain the memory efficiency.

Verifier Agent The Verifier Agent serves as an early-stopping and reflection mechanism, allowing for a balance between effectiveness and efficiency:

$$h, c = VA(q, a) \quad (7)$$

Where q denotes the sub-problems from the Planning Agent, a denotes the answer from the answer

Backbone	Method	StableToolBench													
		I1-Inst.		I1-Cat.		I1-Tool.		I2-Cat.		I2-Inst.		I3-Inst.		Average	
		Pass	Win	Pass	Win	Pass	Win	Pass	Win	Pass	Win	Pass	Win	Pass	Win
GPT-3.5 Turbo	ReACT	41.6 $\pm$ 1.2	/	48.4 $\pm$ 0.5	/	52.5 $\pm$ 0.5	/	52.2 $\pm$ 1.0	/	31.6 $\pm$ 1.2	/	39.9 $\pm$ 2.0	/	44.4 $\pm$ 1.1	/
GPT-3.5 Turbo	DFSDT	54.1 $\pm$ 1.0	64.4	60.1 $\pm$ 0.0	61.4	59.9 $\pm$ 1.7	53.8	60.9 $\pm$ 0.9	62.9	52.8 $\pm$ 3.7	66.0	44.3 $\pm$ 4.8	54.1	55.4 $\pm$ 2.0	60.4
GPT-3.5 Turbo	Smurfs	60.3$\pm$1.5	65.0	67.0$\pm$1.0	69.9	60.3$\pm$1.3	54.4	54.3$\pm$0.4	63.7	42.6$\pm$1.6	64.2	60.1$\pm$1.0	57.4	57.4$\pm$1.1	62.4
Mistral-7B	ReACT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Mistral-7B	DFSDT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Mistral-7B	Smurfs	76.3$\pm$0.8	63.8	86.7$\pm$1.2	62.7	81.0$\pm$1.9	58.2	70.4$\pm$2.7	54.0	63.8$\pm$2.4	67.0	85.2$\pm$0.7	57.4	77.2$\pm$1.6	60.5
GPT-4 Turbo	ReACT	41.1 $\pm$ 1.5	60.1	53.2 $\pm$ 1.3	62.1	42.2 $\pm$ 1.1	48.1	50.0 $\pm$ 0.7	57.3	38.7 $\pm$ 0.8	65.1	37.7 $\pm$ 1.3	47.5	43.8 $\pm$ 1.1	56.7
GPT-4 Turbo	DFSDT	52.7 $\pm$ 1.4	69.9	58.2 $\pm$ 0.9	66.0	59.7 $\pm$ 1.2	58.2	59.3 $\pm$ 0.7	62.1	52.2 $\pm$ 2.3	67.9	61.5 $\pm$ 1.8	65.6	57.3 $\pm$ 1.4	65.0
GPT-4 Turbo	Smurfs	59.3 $\pm$ 1.4	71.2	73.3 $\pm$ 1.3	72.5	67.4 $\pm$ 0.7	69.6	66.7 $\pm$ 1.9	73.4	55.5 $\pm$ 1.4	66.0	70.5 $\pm$ 0.0	72.1	65.5 $\pm$ 1.1	70.8

Table 3: The open-end tool planning task evaluation on the StableToolBench benchmark (Guo et al., 2024). The most effective approach is highlighted in bold, while the second-best is underlined. Win rate is calculated by comparing each model with ChatGPT-ReACT. A win rate higher than 50% means the model performs better than ChatGPT-ReACT.

agent, h and c denotes hint and check status respectively. If check status generated is 0, that means the Verifier Agent thinks the sub-problem isn’t completed, the system will add the thought and answer of this time to the local and global memory, set $t=t+1$ and continue the inference procedure. If check status is 1, the sub-problem is thought to be solved and the system will start to deal with the next sub-problem.

4 Experiments

To evaluate both the effectiveness and efficiency of the Smurfs framework, in this section, we carried out two multi-tool planning tasks: (1) an open-ended task, *StableToolBench* (Guo et al., 2024), and (2) a closed-ended task, *HotpotQA* (Yang et al., 2018). In addition to these main experiments designed to assess the entire framework, we conducted an ablation studies followed by a case study to test the capabilities of each component within the multi-agent framework and investigate the underlying reasons for its effectiveness.

4.1 Open-ended Task: StableToolBench

StableToolBench is a tool learning benchmark derived from ToolBench (Qin et al., 2024), encompassing multi-step tool usage tasks across over 16,000 APIs. The benchmark employs two metrics for evaluation: (1) **Pass Rate** measures the percentage of instructions successfully executed within the allocated budget. (2) **Win Rate** represents the preference selection by a ChatGPT evaluator when presented with two solution paths.

Baselines Following the original paper that introduced the benchmark, we adopt *ReACT* (CoT) (Wei et al., 2023) and *DFSDT* (Touvron et al., 2023) as baseline methods for comparison. Additionally, we include the backbones used in the paper: gpt-3.5-

turbo-0613 (GPT-3.5 Turbo) (OpenAI) and gpt-4-turbo-preview (GPT-4 Turbo). To explore the adaptability of the tool-planning methods, we also include Mistral-7B-Instruct-v0.2 (Mistral-7B) (Jiang et al., 2023) as one of the selected backbones in our experiments.

Settings To minimize the influence of varying tool APIs on experimental results, we conducted all experiments using the same API cache (Guo et al., 2024). For a fair comparison among the candidate methods and to reduce variability, each model was executed once and evaluated three times, with results averaged. Other settings follow those specified in the original benchmark paper.

Results Table 3 displays the results on StableToolBench. For the untrained LLM, Mistral-7B, existing agent frameworks did not improve its performance in tool planning tasks; Mistral-7B failed these tasks when integrated with the ReACT and DFSDT frameworks*. However, Smurfs exhibited exceptional performance: when combined with Mistral-7B, Smurfs achieved competitive scores among the baselines. Through its task decomposition mechanism, Smurfs transforms long-context tasks into simpler ones, enabling the untrained model to effectively utilize external tools for managing complex tasks. Regarding closed-source models, specifically GPT4 in these experiments, Smurfs also demonstrated outstanding performance on the benchmark compared to other agent frameworks and achieved state-of-the-art results on the benchmark. Its high success rate suggests that Smurfs is more effective at finding optimal solution paths compared to ChatGPT.

*Experiment results show that Mistral-7B failed to correctly execute the ‘finish’ action during inference, resulting in invalid responses.

Further Analysis We conducted a detailed analysis of the token costs associated with each tool planning method for the tasks, a critical evaluation aspect for multihop reasoning tasks. As shown in Table 1 (detailed in Appendix G), the average token costs per question and API request are evaluated for ReACT, DFSDT, and Smurfs on StableToolBench. The analysis reveals that DFSDT generally requires about 20,000 tokens per question, encompassing both prompt and completion tokens. This is nearly three times the token cost compared to ReACT and twice as much as Smurfs. Despite this higher token cost, DFSDT does not demonstrate commensurate effectiveness improvements over other methods. These findings underscore the cost-efficiency of the proposed MAS framework, Smurfs, which not only reduces token expenditure in solving multihop planning tasks but also delivers outstanding performance in evaluations.

Backbone	Method	Single-Agent Multi-Agent	HotpotQA			
			Easy	Medium	Hard	All
GPT-3.5 Turbo	CoT		48.21	44.52	34.22	42.32
	Zero-Shot Plan		50.71	45.17	38.23	44.70
Mistral-7B Instruct-v0.2	CoT		33.70	22.38	22.14	26.07
	ReAct		38.09	27.57	22.05	29.24
	Chameleon		37.07	26.67	19.20	27.65
	Reflexion		40.78	35.02	28.36	34.72
	BOLAA		40.86	32.11	22.36	31.78
	ReWOO		38.42	31.89	25.98	32.10
	Smurfs (ours)		<u>45.94</u>	40.74	<u>30.72</u>	39.13
	FireAct		45.52	32.02	30.17	35.90
	AUTOACT		48.69	<u>36.65</u>	31.37	<u>38.89</u>
Llama-2 13B-chat	CoT		37.90	25.28	21.64	28.27
	ReAct		28.68	22.15	21.69	24.17
	Chameleon		40.01	25.39	22.82	29.41
	Reflexion		44.43	37.50	28.17	36.70
	BOLAA		33.23	25.46	25.23	27.97
	ReWOO		30.09	24.01	21.13	25.08
	Smurfs (ours)		<u>42.62</u>	<u>27.21</u>	<u>22.92</u>	<u>30.92</u>
	FireAct		<u>45.83</u>	<u>38.94</u>	26.06	<u>36.94</u>
	AUTOACT		47.29	41.27	32.92	40.49
Llama-2 70B-chat	CoT		45.37	36.33	32.27	37.99
	ReAct		39.70	37.19	33.62	36.83
	Chameleon		46.86	38.79	34.43	40.03
	Reflexion		48.01	46.35	35.64	43.33
	BOLAA		46.44	37.29	33.49	39.07
	ReWOO		42.00	39.58	35.32	38.96
	Smurfs (ours)		<u>52.86</u>	50.77	44.87	49.50
	FireAct		50.82	41.43	35.86	42.70
	AUTOACT		56.94	<u>50.12</u>	<u>38.35</u>	<u>48.47</u>

Table 4: The closed-end tool planning evaluation on HotpotQA (Yang et al., 2018), with some results derived from (Qiao et al., 2024). The most effective approach for each group is highlighted in bold, while the second-best is underlined. Methods marked with  require model training.

4.2 Closed-ended Task: HotpotQA

Compared to open-ended tasks, closed-ended tasks provide a more stable and robust evaluation. To this end, we evaluate the methods on HotpotQA (Yang et al., 2018) in addition to StableToolBench. Hot-

potQA is a multi-hop QA task that is challenging due to the requirement for rich background knowledge, with answers typically being short entities or yes/no responses.

Baselines The compared baselines include CoT (Wei et al., 2023), REACT (Yao et al., 2022), Chameleon (Lu et al., 2023), Reflexion (Shinn et al., 2023), BOLAA (Liu et al., 2023), ReWOO (Xu et al., 2023), FIREACT (Chen et al., 2023a), AutoAct (Qiao et al., 2024).

Settings and Metrics Following the settings in (Qiao et al., 2024), we use open-source Llama-2 models (Touvron et al., 2023) and Mistral-7B (Jiang et al., 2023) as the backbones of each agent to evaluate the performance of Smurfs. The evaluation metrics is reward $\in [0, 1]$, defined as the F1 score grading between the prediction and ground-truth answer. For more details about the experiment, see Appendix D.

Results Smurfs, as an untrained MAS system, not only comprehensively outperforms untrained agents but also achieves and even surpasses the accuracy of trained agents across most backbone models. This sufficiently demonstrates that the mechanism of smurfs ensures strong generalization capabilities while maintaining high effectiveness.

Observations indicate that the performance of Llama-2-13b-chat on smurfs-related tasks is sub-optimal, likely due to its limited capabilities in tool arguments generation. Specifically, the primary issue identified is that, when the Executor agent successfully selects relevant tool, it tends to produce hallucination arguments that can’t be used by the tools. This indicates that Llama-2-13b-chat may need further training for usage of tools. The experiment results may substantiate this viewpoint, demonstrating that the untrained methods of llama-2-13b-chat generally exhibit significantly lower accuracy compared to the trained methods. To provide further insight into the model’s performance in the tool planning process, we manually categorized the types of errors made by Smurfs on the hotpotQA hard dataset in appendix E.

5 Ablation Study

5.1 Importance of each component in MAS

We performed an ablation study to investigate the impact of each agent in our framework. We removed each agent individually, except for the indis-

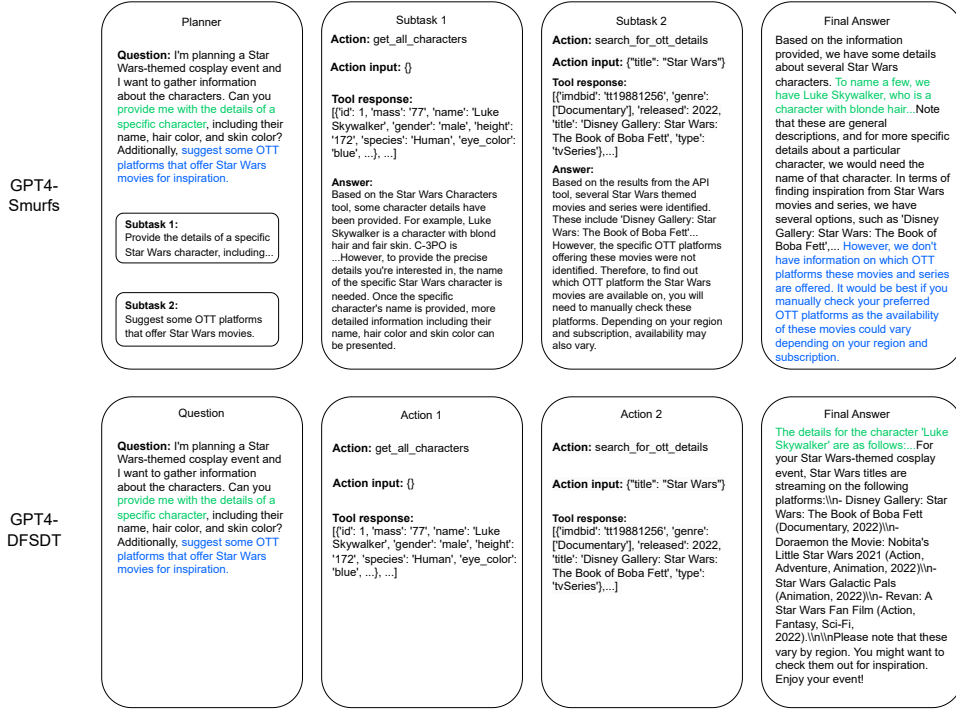


Figure 3: The illustration of how GPT4-Smurfs and GPT4-DFSdT solve long context problem. The two sub-questions and their corresponding answers are marked in two colors.

	I3-Inst.	
	Pass (%)	Win (%)
GPT-3.5 Turbo with Smurfs	60.1 $\pm$ 1.0	57.4
w/o Answer Agent	57.4 $\pm$ 2.9	49.2
w/o Verifier Agent	54.1 $\pm$ 2.7	42.6
w/o Planning Agent	35.5 $\pm$ 3.3	42.6
w/o Planning & Verifier Agent	58.5 $\pm$ 2.0	57.4
GPT-4 Turbo with Smurfs	70.5 $\pm$ 1.0	72.1
w/o Answer Agent	82.2 $\pm$ 2.5	72.1
w/o Verifier Agent	79.2 $\pm$ 0.8	63.9
w/o Planning Agent	71.9 $\pm$ 2.8	63.9
w/o Planning & Verifier Agent	79.8 $\pm$ 2.8	67.2

Table 5: Ablation study on StableToolBench I3-Inst subset to investigate the importance of each component within the framework.

pensible Executor Agent, and compared the results to the complete framework.

Settings (1) **Verifier Agent Removal:** Without Verifier Agent, the framework uses a general DFSdT in each subtask, i.e. including a finish tool into the tool library and deciding whether to stop at tool choice step. (2) **Answer Agent Removal:** Without Answer Agent, the framework uses full tool response instead of the summary of tool response in its memory. (3) **Planning Agent Removal:** Without Planning Agent, the framework

uses Verifier Agent to decide whether the task is completed. (4) **Planning & Verifier Agent Removal:** Without Planning and Verifier Agent, the only difference between Smurfs and DFSdT will be the Answer Agent, i.e. including a finish tool in the tool library, deciding whether to stop at tool choice step and using summary of tool response in its memory.

Results Table 5 shows the experiment result, highlighting several key insights regarding the impact of different components in the multi-agent system.

First, the removal of any component generally reduces the win rate, highlighting each component’s significance. Second, performance degradation trends remain consistent: Eliminating the Answer Agent has minimal impact, whereas removing the Planning Agent causes the greatest decline. However, this does not imply the relative importance of these agents but rather suggests that the Verifier-Planning Agent workflow is more robust than the Verifier-Answer Agent workflow.

Third, the performance impact varies by model capability. For GPT-3.5, the removal of the Answer Agent reduces both the pass and the win rates, while for GPT-4, it preserves the win rate and im-

proves the pass rate, probably due to the superior long-text reasoning ability of GPT-4, which processes additional tool response more effectively. Removal of the Planning Agent significantly reduces the pass rate of GPT-3.5 but has a minimal effect on GPT-4, suggesting that the GPT-4 Verifier Agent is more robust. **These findings may indicate that more powerful models can compensate for agent removal, sometimes even benefiting from additional context, while weaker models benefit more from complex workflow and compressed context.**

We also noticed an interesting phenomenon: when only the Planning Agent is removed, the system experiences a major decline in performance, while removing the Verifier Agent in addition to the Planning Agent improves performance again. This may indicate that the Verifier Agent’s ability to pause tasks or provide next-step guidance is suboptimal and requires further training.

In conclusion, our findings offer some insight into the relationship between model capabilities and multi-agent system performance. The optimal number of agents and workflow may vary depending on the capacity of the model. **We propose the hypothesis that weaker models benefit more from complex multi-agent systems and context segmentation, while stronger models perform better with comprehensive context and simpler agent systems.** To validate this hypothesis, more extensive ablation studies are needed under a wider range of models and constraints to explore the influence of different context structures and reasoning workflows. We leave this investigation for future work.

5.2 Case Study

As shown in Figure 3, although GPT4-DFSĐT and GPT4-Smurfs use the same tool calls to solve the problem, GPT4-DFSĐT only answers the first sub-question correctly while GPT4-Smurfs answers both sub-questions accurately. In the process of addressing the second sub-question, it is notable that the tool response only mentions titles of film and television products related to "Star Wars", without addressing OTT platforms. GPT-4-DFSĐT erroneously interprets these titles as responses to the question, while GPT-4-Smurfs adeptly identifies this discrepancy and provides a more appropriate response. This case highlights that in situations where tool responses are lengthy and questions are complex, the single agent framework like DFSĐT

may be susceptible to distractions from irrelevant information, leading to erroneous answers. Conversely, the context-efficient Smurfs framework demonstrates a reduced susceptibility to irrelevant information, thereby generating more accurate answers.

6 Conclusion

In this paper, we propose *Smurfs*, an innovative MAS framework designed to enhance the tool-planning capabilities of LLMs without requiring additional training. Through extensive experiments on both open-ended and closed-ended tool planning benchmarks, Smurfs demonstrate its effectiveness by consistently outperforming baseline methods. Ablation study further provides deeper insights into the impact of each agent in our framework. Based on the ablation study, we propose a hypothesis that can be further tested through more comprehensive experiments in future research: weaker models benefit more from complex multi-agent systems and context segmentation, while stronger models perform better with comprehensive context and simpler agent systems. The findings not only advance the state-of-the-art in multi-tool planning systems but also highlight the potential of modular, training-free frameworks for LLMs in various practical applications.

Looking forward, future research could focus on exploring Smurfs’ use in new domains, such as facilitating the synthesis of high-quality multi-tool planning data and enhancing the base model’s reasoning and tool-use abilities, further advancing the field of adaptive AI systems.

Limitations

Model Size Constraints: Due to computational constraints, our experiments did not include larger and more diverse types of LLMs. We believe this would not affect the main observations of this paper.

Agent Component Scale-Up: Although we selected the most common and intuitive agent roles for the proposed MAS, there are many possibilities for researchers to explore. Investigating more well-designed agent roles may help improve the effectiveness of the agent system, and developing automated methods to identify these roles could facilitate effective scaling.

Acknowledging these limitations, future research should aim to address these gaps to provide

a more comprehensive understanding of the Smurfs framework’s capabilities and potential areas for improvement.

Acknowledgements

This work was supported by the Shenzhen Science and Technology Program (JCYJ20220818103001002), Shenzhen Doctoral Startup Funding (RCBS20221008093330065), Tianyuan Fund for Mathematics of National Natural Science Foundation of China (NSFC) (12326608), Shenzhen Key Laboratory of Cross-Modal Cognitive Computing (grant number ZDSYS20230626091302006), and Shenzhen Stability Science Program 2023.

References

- Stanley H Ambrose. 2001. Paleolithic technology and human evolution. *Science*, 291(5509):1748–1753.
- Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. 2023a. [Fire-act: Toward language agent fine-tuning](#). *Preprint*, arXiv:2310.05915.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2023b. [Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors](#). *Preprint*, arXiv:2308.10848.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. 2019. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*.
- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359.
- Yicheng Fu, Raviteja Anantha, and Jianpeng Cheng. 2024. [Camphor: Collaborative agents for multi-input planning and high-order reasoning on device](#). *Preprint*, arXiv:2410.09407.
- Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. [Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models](#). *Preprint*, arXiv:2403.07714.
- Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. 2023. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. [Camel: Communicative agents for "mind" exploration of large language model society](#). *Preprint*, arXiv:2303.17760.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Zhiwei Liu, Weiran Yao, Jianguo Zhang, Le Xue, Shelby Heinecke, Rithesh Murthy, Yihao Feng, Zeyuan Chen, Juan Carlos Niebles, Devansh Arpit, Ran Xu, Phil Mui, Huan Wang, Caiming Xiong, and Silvio Savarese. 2023. [Bola: Benchmarking and orchestrating llm-augmented autonomous agents](#). *Preprint*, arXiv:2308.05960.
- Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. [Chameleon: Plug-and-play compositional reasoning with large language models](#). *Preprint*, arXiv:2304.09842.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*.
- Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2024. [Learning to compress prompts with gist tokens](#). *Preprint*, arXiv:2304.08467.
- Kenneth Page Oakley and London British Museum. 1972. *Man the tool-maker*. 538. British Museum (Natural History) London.
- OpenAI. ChatGPT. <https://openai.com/blog/chatgpt>.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. [Generative agents: Interactive simulacra of human behavior](#). *Preprint*, arXiv:2304.03442.
- Fabio Petroni, Patrick Lewis, Aleksandra Piktus, Tim Rocktäschel, Yuxiang Wu, Alexander H Miller, and Sebastian Riedel. 2020. How context affects language models’ factual predictions. *arXiv preprint arXiv:2005.04611*.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. [Chatdev: Communicative agents for software development](#). *Preprint*, arXiv:2307.07924.

- Shuofei Qiao, Ningyu Zhang, Runnan Fang, Yujie Luo, Wangchunshu Zhou, Yuchen Eleanor Jiang, Chengfei Lv, and Huajun Chen. 2024. [Autoact: Automatic agent learning from scratch for qa via self-planning](#). *Preprint*, arXiv:2401.05268.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, dahai li, Zhiyuan Liu, and Maosong Sun. 2024. [ToolLLM: Facilitating large language models to master 16000+ real-world APIs](#). In *The Twelfth International Conference on Learning Representations*.
- Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, 11:1316–1331.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. [Hugging-gpt: Solving ai tasks with chatgpt and its friends in hugging face](#). *Preprint*, arXiv:2303.17580.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. 2023. [Large language models can be easily distracted by irrelevant context](#). *Preprint*, arXiv:2302.00093.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). *Preprint*, arXiv:2303.11366.
- Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, Ye Tian, and Sujian Li. 2023. [Restgpt: Connecting large language models with real-world restful apis](#). *Preprint*, arXiv:2306.06624.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. [Chain-of-thought prompting elicits reasoning in large language models](#). *Preprint*, arXiv:2201.11903.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. [Autogen: Enabling next-gen llm applications via multi-agent conversation](#). *Preprint*, arXiv:2308.08155.
- Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. 2023. [Rewoo: Decoupling reasoning from observations for efficient augmented language models](#). *Preprint*, arXiv:2305.18323.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Da Yin, Faeze Brahman, Abhilasha Ravichander, Khyathi Chandu, Kai-Wei Chang, Yejin Choi, and Bill Yuchen Lin. 2024. [Agent lumos: Unified and modular training for open-source language agents](#). *Preprint*, arXiv:2311.05657.
- Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Ren Kan, Dongsheng Li, and Deqing Yang. 2024. Easytool: Enhancing llm-based agents with concise tool instruction. *arXiv preprint arXiv:2401.06201*.
- Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. 2023. [Least-to-most prompting enables complex reasoning in large language models](#). *Preprint*, arXiv:2205.10625.

A Comparison between Smurfs and existing MAS

In conclusion, Smurfs stands out compared to existing MAS in followig terms:

Flexibility Smurfs utilize enhanced version of DFSDT, which intergrades global and iterative planning, while existing MAS mainly use ReAct and iterative planning. This make Smurfs more flexible in the planning process.

Adaptability Smurfs realize superior performance on multiple different benchmarks, proving the adaptability of Smurfs. Existing MAS like RestGPT, Lumos are tailored for specific downstream task and need additional training to be used in other scenario.

Learning Efficiency Smurfs do not need training, thus have the highest learning efficiency among existing MAS. It only need query and tool documentation from the user.

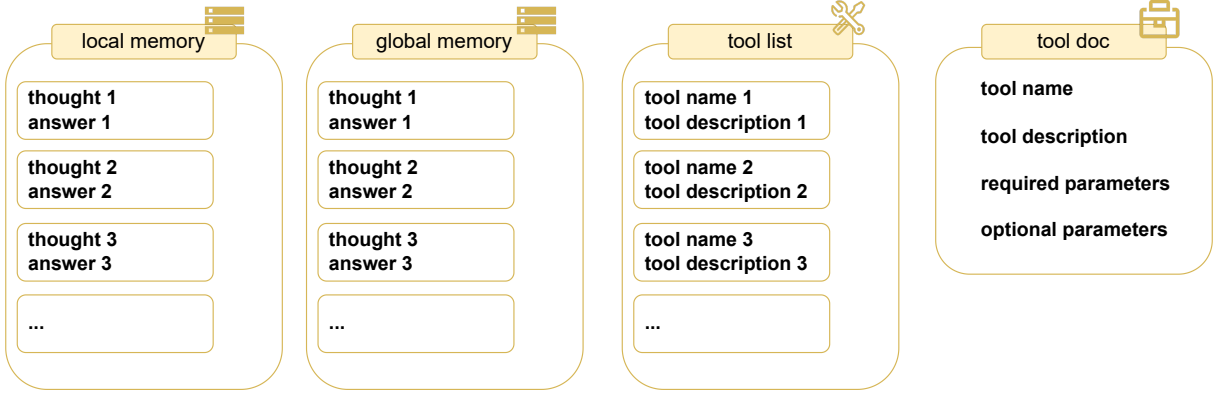


Figure 4: Demonstration of the memory of the Smurfs framework.

To further illustrate these points, we conducted a detailed comparison between Smurfs and two well-known Multi-Agent Systems, highlighting their differences and the adjustments required when learning out-of-box tasks.

CAMEL CAMEL (Li et al., 2023) is a communicative agent framework. It uses role-play technique and inception-prompting technique to achieve autonomous cooperation between agents. CAMEL does not natively support tool use settings. CAMEL is considered to perform poorly when generalizing to new tasks (e.g., on the MATH dataset (Wu et al., 2023)).

Autogen Autogen (Wu et al., 2023) is not designed as an agent framework for any specific task scenario. Instead, it provides a multi-agent conversation framework that allows users to customize agent characteristics and complete tasks through discussions among different agents. Autogen addresses various problems not by employing a uniform workflow, but by allowing users to design customized agents and workflows flexibly based on their tasks.

Smurfs Smurfs is designed as a unified workflow specifically for complex multi-tool planning scenarios. For different tasks, Smurfs only needs to adjust the few-shot examples in the agent prompts and provide documentation for the tools applicable to the task, allowing Smurfs to generalize to other task scenarios. Smurfs was initially designed for the Stable-toolbench, which itself encompasses various types of tasks and has access to over 16,000 plugins. HotpotQA was subsequently introduced to evaluate the performance of Smurfs on closed-end tasks. When migrating from Stabletoolbench to HotpotQA, only the few-shot examples in the plan-

ning agent prompts were modified, along with the provision of plugin documentation for HotpotQA. The rest of the system continued using the same unified framework as in Stabletoolbench. Experiment results demonstrate that our untrained out-of-box unified framework achieves and even surpasses performance of agent systems specifically trained on HotpotQA, such as Autoact and Fireact, showcasing Smurfs’ flexibility in generalization.

B Details of DFSDT

DFSDT (Qin et al., 2024) gives control to the model to stop and rollback the solution trajectory by using Finish tool, thus addressing limitations of ReACT. Finish tool has two parameters **give answer**: model thinks the task is finished and decide to give answer and **give up and restart**: model thinks current trajectory can’t lead to correct answer and decide to rollback.

C Details of the Smurfs

Executor Agent Details As illustrated in Figure 5, given a sub-problem p , Executor Agent first thinks about what to do this time, generates thought γ according to p , local memory M , hint h from the Verifier Agent and tool list at the current step τ . Then it will choose action α to complete the sub-problem using p , γ and τ . After that, parameters of α are generated using p , local memory M and tool description of the action $D[\alpha]$. Tool is then invoked to complete the task.

Memory Details As illustrated in Figure 4, there are four kinds of memory in Smurfs. Local memory stores thought-answer pairs of the current solution trajectory, while global memory stores all history solution trajectory (including those that is backtracked). Tool list only stores available tools’ name

Compression Method	Applicable Scenario	Compressed Object	Implementation
Gist Tokens (Mu et al., 2024)	General scenarios	Frequently used system prompts	Training LLM to compress system prompts, reducing token usage.
CAMPBOR (Fu et al., 2024)	Tool use scenarios	Tool descriptions	Adopts a similar approach to gist tokens, compressing each tool description into a single token, thus reducing token cost.
Smurfs	Complex multi-tool planning scenarios	Input context in tool planning	Operates on multi-tool planning workflows, compressing the context needed for each tool planning process.

Table 6: Comparison of Token Compression Methods

and its usage description, while tool doc stores all detailed information about the tools including parameters details. Through using different kinds of memory under different circumstances, Smurfs can use DFSDT in a context efficient way.

Restart Mechanism Every time Smurfs generate an intermediate output, a format checker is used to check whether the output is of the expected format. If not, Smurfs will retry the same step until reach retry limit or generate correct format output. This mechanism is used in addition to the rollback mechanism to handle the situation where the system can generate correct content but fail to follow the output format.

D Experiment Settings for Hotpot QA

Following settings in (Qiao et al., 2024), which is randomly select 300 dev questions divided into three levels for evaluation, with 100 questions in each level. For tool library that can be used in HotpotQA, see Table 7

E Error Analysis on HotpotQA

To provide further insight into the model performance in the tool planning process, we manually categorized the types of error made by Smurfs in the hotpotQA hard dataset. Table 8 shows that the most frequent errors committed by mistral-7b Smurfs is tool argument fail, followed by bad planning and answer miss. Smurfs do not make tool choice errors and premature termination errors. This shows that Smurfs actually alleviates the premature termination problem, making the tool choice process more robust. Additionally, we note that a portion of the error samples are false negatives which arise when the generated answers differ in expression from the ground truth but are

equivalent in meaning. This highlights potential directions for future improvements in Smurfs.

F Prompts for multi-agent implementation

Prompts used by each agent and their example outputs are shown in Figure 6 to 12.

G Token Cost on StableToolBench Evaluation

We analyzed the token cost for the StableToolBench experiments. As shown in Table 9, the total token cost for each subtask within the StableToolBench is compared across three candidate tool-planning methods. The data demonstrates that, across all tasks from easy to hard, DFSDT consistently incurs high token costs, while the other two methods maintain relatively low token costs. This verifies the context-efficiency of the proposed method.

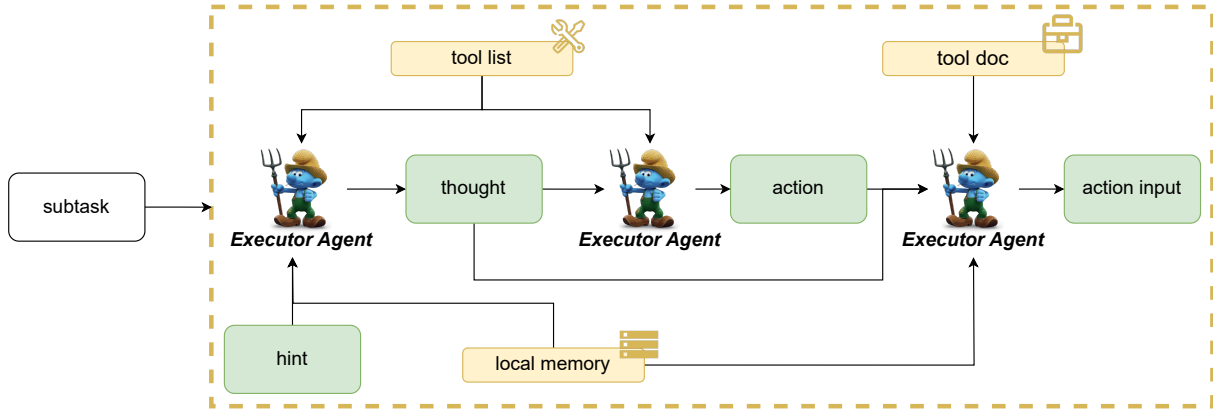


Figure 5: Details of the executor agent working process

Name	Definition	Usage
BingSearch	BingSearch engine can search for rich knowledge on the internet based on keywords, which can compensate for knowledge fallacy and knowledge outdated.	BingSearch[query], which searches the exact detailed query on the Internet and returns the relevant information to the query. Be specific and precise with your query to increase the chances of getting relevant results. For example, Bingsearch[popular dog breeds in the United States]
Retrieve	Retrieve additional background knowledge crucial for tackling complex problems. It is especially beneficial for specialized domains like science and mathematics, providing context for the task	Retrieve[entity], which retrieves the exact entity on Wikipedia and returns the first paragraph if it exists. If not, it will return some similar entities to retrieve. For example, Retrieve[Milhouse]
Lookup	A Lookup Tool returns the next sentence containing the target string in the page from the search tool, simulating Ctrl+F functionality on the browser.	Lookup[keyword], which returns the next sentence containing the keyword in the last passage successfully found by Retrieve or BingSearch. For example, Lookup[river].

Table 7: Tool library for HotpotQA.

	Bad Planning	Answer Miss	Tool Wrong	Tool Argument Fail	False Negative	Premature Termination	Total Accuracy
Mistral-7b-Smurfs	11	7	0	28	12	0	0.42

Table 8: Error analysis for Smurfs on HotpotQA Hard.

Backbone	Method	I1-Inst.		I1-Cat.		I1-Tool.		StableToolBench I2-Cat.		I2-Inst.		I3-Inst.		Average	
		Total	Avg.	Total	Avg.	Total	Avg.	Total	Avg.	Total	Avg.	Total	Avg.	Total	Avg.
GPT-3.5 Turbo	ReACT	1,010,304	6,198	824,676	5,390	1,010,514	6,396	900,855	7,265	824,510	7,778	461,121	7,559	838,663	6,764
GPT-3.5 Turbo	DFSDT	3,303,062	20,264	2,745,667	17,945	3,152,532	19,953	2,560,297	20,648	3,098,365	29,230	1,390,787	22,800	2,708,452	21,807
GPT-3.5 Turbo	Smurfs	1,090,404	7,127	1,917,348	11,763	1,464,535	9,269	957,088	7,638	1,096,162	10,341	632,084	10,362	1,191,270	9,417

Table 9: Token costs for various candidate tool-planning methods on the StableToolBench benchmark (Guo et al., 2024). ‘Total’ indicates the total number of tokens used to complete each subtask, including both prompt and completion tokens. ‘Avg.’ represents the average number of tokens used per question within the subtasks. Higher token counts imply greater costs for solving the same task.

Planning Agent

Prompt:

You need to decompose a complex user's question into some simple sub-tasks and let the model execute it step by step. Please note that:

1. You should only decompose this complex user's question into some simple sub-tasks which can be executed easily by using a single tool.
2. Each simple subtask should be expressed into natural language.
3. Each subtask should contain the necessary information from the original question and should be complete, explicit and self-consistent.
4. You must ONLY output in a parsible JSON format. An example output looks like:

```
""  
{ "Tasks": ["Task 1", "Task 2", ...] }  
""
```

This is the user's question: I'm planning a trip to Turkey and need information about postal codes in Istanbul. Can you provide me with the postal code and district for Istanbul province with plate number 34? Additionally, I would like to know if there are any transit agencies available in Istanbul. Please fetch their names and contact numbers.

Output: "Tasks": ["Find the postal codes and districts for plate number 34 in Istanbul.", "Search for transit agencies and their contact numbers in Istanbul."]

This is the user's question: I recently moved to a new address and I need to update my information. Can you retrieve my address details using the postal code 75094080? Additionally, I would like to know the companies that offer shipping services.

Output: {"Tasks": ["retrieve the address details using the postal code 75094080", "search for companies that offer shipping services to my address"]}

This is the user's question: {question}

Output:

Example Output:

```
{ "Tasks": ["Determine the postal code and district for Istanbul province with plate number 34.", "Find out if there are any transit agencies in Istanbul.", "Get the names of the transit agencies in Istanbul.", "Obtain the contact numbers for the transit agencies in Istanbul." ] }
```

Figure 6: An example prompt for task decomposition in the framework.

Tool Check

Prompt:

As a powerful language model, you're equipped to answer user's question with accumulated knowledge.

However, in some cases, you need to use external APIs to answer accurately.

Thus, you need to check whether the user's question requires you to call an external API to solve it.

Here are some tips to help you check:

1. If the user's question requires real-time information, since your knowledge base isn't updated in real-time, any such question will demand an API call.
2. If you need to obtain information (e.g., ID, name, phone number, geographical location, rank, etc.), you need to call the database APIs if you are not sure.
3. If the question demand a database search or internet research to generate an answer, this is another situation where an API call is necessary.

If need, please output 'YES'; If not, please output 'NO'

You need to give reasons first and then decide whether to keep it or not. You must only output in a parsible JSON format.

Two example outputs look like:

Example 1: {"Reason": "The reason why you think you do not need to call an external API to solve the user's question", "Choice": "No"}

Example 2: {"Reason": "The reason why you think you need to call an external API to solve the user's question", "Choice": "Yes"}

This is the user's question:

{question}

Output:

Example Output:

{"Reason": "To determine the postal code and district for a specific location based on a plate number, we would typically need to access a combination of databases, including vehicle registration databases and postal code databases. Since we do not have direct access to these databases, we will need to call external APIs to retrieve this information. Therefore, the user's question requires an API call.", "Choice": "Yes"}

Figure 7: An example prompt for tool check in the framework.

Executor Agent: Thought

Prompt:

You need to analyse the previous execution history and generate your internal reasoning and thoughts on the task, and how you plan to solve it based on the current attempts.

Do not output thought that is too long. Output in 2-3 sentences is OK.

This is the user's task:

{task}

This is the Tool List:

{functions}

This is the previous execution history:

{messages}

This is the hint comes from the evaluator:

{hint}

Output:

Example Output:

Looking at the task, the user wants to find the postal codes and districts for the number plate 34 in Istanbul. I have a tool that provides Turkish plates, and since Istanbul is in Turkey, this tool might provide the needed information. I will proceed to use the 'Logistics:Turkey Postal Codes:il' tool to accomplish the task.

Figure 8: An example prompt for tool check in the framework.

Executor Agent: Action

Prompt:

This is the user's question:

{question}

These are the tools you can select to solve the question:

Tool List:

{Tool_list}

Please note that:

1. You should only choose one tool from the Tool List to solve this question.
2. You must ONLY output the ID of the tool and your reason for choosing it in a parsible JSON format. An example output looks like:

'''

Example: {"ID": ID of the tool, "Reason": The reason for choosing the tool}

'''

Output:

Example Output:

{'ID': '1', 'Reason': 'This tool provides information about Turkish plates and postal codes, which can be applied to the given task of finding the postal codes and districts for the number plate 34 in Istanbul.'}

Figure 9: An example prompt for action generation in the framework.

Executor Agent: Action Input

Prompt:

Given a user's question and a API tool documentation, you need to output parameters according to the API tool documentation to successfully call the API to solve the user's question.

Please note that:

1. The Example in the API tool documentation can help you better understand the use of the API.
2. Ensure the parameters you output are correct. The output must contain the required parameters, and can contain the optional parameters based on the question. If no parameters in the required parameters and optional parameters, just leave it as {}
3. If the user's question mentions other APIs, you should ONLY consider the API tool documentation I give and do not consider other APIs.
4. The question may have dependencies on answers of other questions, so we will provide logs of previous questions and answers for your reference.
5. You must ONLY output in a parsible JSON Format. The example output looks like:

'''

Example: {"keyword": "Artificial Intelligence", "language": "English"}

'''

There are logs of previous questions and answers:

previous_log

This is the current user's question: question

This is API tool documentation: api_dic

Output:

Example Output:

{"il": 34}

Figure 10: An example prompt for action input generation in the framework.

Answer Agent

Prompt:

You should answer the question based on the response output by the API tool.

Please note that:

1. Answer the question in natural language based on the API response reasonably and effectively.
2. The user cannot directly get API response, so you need to make full use of the response and give the information in the response that can satisfy the user's question in as much detail as possible.
3. Do not output answer that is too long. Output in 3-6 sentences is OK.

This is the user's question:

{question}

This is the API response:

{call_result}

Output:

Example Output:

The postal codes and districts for the plate number 34 in Istanbul include: Adalar district with postal code 34975 for Burgazada area, 34970 for Büyükada area, 34973 for Heybeliada area and 34977 for Kınalıada area. There is also the Arnavutköy district with postal code 34275 for areas like Anadolu, Arnavutköy Merkez, İmrahor, İslambey, Mustafa Kemal Paşa, Nenehatun, and Yavuz Selim. Another area in Arnavutköy district is Baklalı with postal code 34277.

Figure 11: An example prompt for Answer Agent in the framework.

Verifier Agent

Prompt:

An agent is trying to solve the query proposed by the user.

You need to evaluate whether the given query has been completed reasonably and accurately. If so, summarize the solution to the user. If not, summarize the current progress, and propose what is missing.

Your response contains following elements:

Speak: (your words to the agent if the task is pending, or a complete answer based on the full execution log to the user if the task is finished)

Status: (0 or 1. 0 for pending and 1 for finished)

You must only output in a parsible JSON format. Two example outputs look like:

Example 1: {"Speak": "answer based on the full execution log to the user", "Status": "1"}

Example 2: {"Speak": "your words to the group if the task is pending", "Status": "0"}

This is the answer from the previous execution result:

{answer}

This is the original question: {question}

Output:

Example Output:

{"Speak": "The postal codes and districts for plate number 34 in Istanbul include: Adalar district with postal code 34975 for Burgazada area, 34970 for Büyükada area, 34973 for Heybeliada area and 34977 for Kınalıada area. In addition, Arnavutköy district has postal code 34275 for areas such as Anadolu, Arnavutköy Merkez, İmrahor, İslambey, Mustafa Kemal Paşa, Nenehatun, and Yavuz Selim. Another part of Arnavutköy district, Baklalı, has the postal code 34277.", "Status": "1"}

Figure 12: An example prompt for Verifier Agent in the framework.